

Pregel

class NodeBuilder

METHOD	DESCRIPTION
<code>subscribe_only</code>	Subscribe to a single channel.
<code>subscribe_to</code>	Add channels to subscribe to.
<code>read_from</code>	Adds the specified channels to read from, without subscribing to them.
<code>do</code>	Adds the specified node.
<code>write_to</code>	Add channel writes.
<code>meta</code>	Add tags or metadata to the node.
<code>build</code>	Builds the node.

meth subscribe_only

```
subscribe_only(channel: str) -> Self
```



Subscribe to a single channel.

meth subscribe_to

```
subscribe_to(*channels: str, read: bool = True) -> Self
```



Add channels to subscribe to.

Node will be invoked when any of these channels are updated, with a dict of the channel values as input.

PARAMETER	DESCRIPTION
<code>channels</code>	Channel name(s) to subscribe to TYPE: <code>str</code> DEFAULT: <code>()</code>
<code>read</code>	If <code>True</code> , the channels will be included in the input to the node. Otherwise, they will trigger the node without being sent in input. TYPE: <code>bool</code> DEFAULT: <code>True</code>

RETURNS	DESCRIPTION
<code>Self</code>	Self for chaining

meth `read_from`

```
read_from(*channels: str) -> Self
```



Adds the specified channels to read from, without subscribing to them.

meth `do`

```
do(node: RunnableLike) -> Self
```



Adds the specified node.

meth `write_to`

```
write_to(*channels: str | ChannelWriteEntry, **kwargs: _WriteValue) -> Self
```



Add channel writes.

PARAMETER	DESCRIPTION
<code>*channels</code>	Channel names to write to.

PARAMETER	DESCRIPTION	
TYPE: <code>str</code> <code>ChannelWriteEntry</code>	DEFAULT: <code>()</code>	
<code>**kwargs</code>	Channel name and value mappings.	
TYPE: <code>_WriteValue</code>	DEFAULT: <code>{}</code>	

RETURNS	DESCRIPTION	
<code>Self</code>	Self for chaining	

meth meta

```
meta(*tags: str, **metadata: Any) -> Self
```



Add tags or metadata to the node.

meth build

```
build() -> PregelNode
```



Builds the node.

class Pregel

Bases: `PregelProtocol[StateT, ContextT, InputT, OutputT]`, `Generic[StateT, ContextT, InputT, OutputT]`

Pregel manages the runtime behavior for LangGraph applications.

Overview

Pregel combines **actors** and **channels** into a single application. **Actors** read data from channels and write data to channels. Pregel organizes the execution of the application into multiple steps, following the **Pregel Algorithm/Bulk Synchronous Parallel** model.

Each step consists of three phases:

- **Plan:** Determine which **actors** to execute in this step. For example, in the first step, select the **actors** that subscribe to the special **input** channels; in subsequent steps, select the **actors** that subscribe to channels updated in the previous step.
- **Execution:** Execute all selected **actors** in parallel, until all complete, or one fails, or a timeout is reached. During this phase, channel updates are invisible to actors until the next step.
- **Update:** Update the channels with the values written by the **actors** in this step.

Repeat until no **actors** are selected for execution, or a maximum number of steps is reached.

Actors

An **actor** is a `PregelNode`. It subscribes to channels, reads data from them, and writes data to them. It can be thought of as an **actor** in the Pregel algorithm. `PregelNodes` implement LangChain's `Runnable` interface.

Channels

Channels are used to communicate between actors (`PregelNodes`). Each channel has a value type, an update type, and an update function – which takes a sequence of updates and modifies the stored value. Channels can be used to send data from one chain to another, or to send data from a chain to itself in a future step. LangGraph provides a number of built-in channels:

BASIC CHANNELS: LASTVALUE AND TOPIC

- `LastValue`: The default channel, stores the last value sent to the channel, useful for input and output values, or for sending data from one step to the next
- `Topic`: A configurable PubSub Topic, useful for sending multiple values between *actors*, or for accumulating output. Can be configured to deduplicate values, and/or to accumulate values over the course of multiple steps.

ADVANCED CHANNELS: CONTEXT AND BINARYOPERATORAGGREGATE

- `Context`: exposes the value of a context manager, managing its lifecycle. Useful for accessing external resources that require setup and/or teardown. e.g. `client = Context(httpx.Client)`
- `BinaryOperatorAggregate`: stores a persistent value, updated by applying a binary operator to the current value and each update sent to the channel, useful for computing aggregates over multiple steps. e.g. `total = BinaryOperatorAggregate(int, operator.add)`

Examples

Most users will interact with Pregel via a [StateGraph \(Graph API\)](#) or via an [entrypoint \(Functional API\)](#).

However, for **advanced** use cases, Pregel can be used directly. If you're not sure whether you need to use Pregel directly, then the answer is probably no - you should use the Graph API or Functional API instead. These are higher-level interfaces that will compile down to Pregel under the hood.

Here are some examples to give you a sense of how it works:



Single node application



```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
    NodeBuilder().subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

app.invoke({"a": "foo"})
```

```
{'b': 'foofoo'}
```

Using multiple nodes and multiple output channels

```
from langgraph.channels import LastValue, EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
    NodeBuilder().subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

node2 = (
    NodeBuilder().subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app = Pregel(
    nodes={"node1": node1, "node2": node2},
    channels={
        "a": EphemeralValue(str),
        "b": LastValue(str),
        "c": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b", "c"],
)

app.invoke({"a": "foo"})
```

```
{'b': 'foofoo', 'c': 'foofoofoofoo'}
```

Using a Topic channel

```
from langgraph.channels import LastValue, EphemeralValue, Topic
from langgraph.pregel import Pregel, NodeBuilder

node1 = (
    NodeBuilder().subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node2 = (
    NodeBuilder().subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app = Pregel(
    nodes={"node1": node1, "node2": node2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)

app.invoke({"a": "foo"})
```

```
{"c": ["foofoo", "foofoofoofoo"]}
```

Using a BinaryOperatorAggregate channel

```
from langgraph.channels import EphemeralValue, BinaryOperatorAggregate
from langgraph.pregel import Pregel, NodeBuilder
```

```
node1 = (
    NodeBuilder().subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)
```

```
node2 = (
    NodeBuilder().subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)
```

```
def reducer(current, update):
    if current:
        return current + " | " + update
    else:
        return update
```

```
app = Pregel(
    nodes={"node1": node1, "node2": node2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
    },
    input_channels=["a"],
    output_channels=["c"],
)
```

```
app.invoke({"a": "foo"})
```

```
{'c': 'foofoo | foofoofoofoo'}
```



Introducing a cycle



This example demonstrates how to introduce a cycle in the graph, by having a chain write to a channel it subscribes to.

Execution will continue until a `None` value is written to the channel.

```
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder, ChannelWriteEntry

example_node = (
    NodeBuilder()
    .subscribe_only("value")
    .do(lambda x: x + x if len(x) < 10 else None)
    .write_to(ChannelWriteEntry(channel="value", skip_none=True))
)

app = Pregel(
    nodes={"example_node": example_node},
    channels={
        "value": EphemeralValue(str),
    },
    input_channels=["value"],
    output_channels=["value"],
)

app.invoke({"value": "a"})
```

```
{'value': 'aaaaaaaaaaaaaaaa'}
```

METHOD	DESCRIPTION
<code>stream</code>	Stream graph steps for a single input.
<code>astream</code>	Asynchronously stream graph steps for a single input.
<code>invoke</code>	Run the graph with a single input and config.
<code>ainvoke</code>	Asynchronously run the graph with a single input and config.
<code>get_state</code>	Get the current state of the graph.
<code>aget_state</code>	Get the current state of the graph.

METHOD	DESCRIPTION
<code>get_state_history</code>	Get the history of the state of the graph.
<code>aget_state_history</code>	Asynchronously get the history of the state of the graph.
<code>update_state</code>	Update the state of the graph with the given values, as if they came from
<code>auupdate_state</code>	Asynchronously update the state of the graph with the given values, as if they came from
<code>bulk_update_state</code>	Apply updates to the graph state in bulk. Requires a checkpoint to be set.
<code>abulk_update_state</code>	Asynchronously apply updates to the graph state in bulk. Requires a checkpoint to be set.
<code>get_graph</code>	Return a drawable representation of the computation graph.
<code>aget_graph</code>	Return a drawable representation of the computation graph.
<code>get_subgraphs</code>	Get the subgraphs of the graph.
<code>aget_subgraphs</code>	Get the subgraphs of the graph.
<code>with_config</code>	Create a copy of the Pregel object with an updated config.

meth stream

```

stream(
    input: InputT | Command | None,
    config: RunnableConfig | None = None,
    *,
    context: ContextT | None = None,
    stream_mode: StreamMode | Sequence[StreamMode] | None = None,
    print_mode: StreamMode | Sequence[StreamMode] = (),
    output_keys: str | Sequence[str] | None = None,
    interrupt_before: All | Sequence[str] | None = None,

```

```

interrupt_after: All | Sequence[str] | None = None,
durability: Durability | None = None,
subgraphs: bool = False,
debug: bool | None = None,
**kwargs: Unpack[DeprecatedKwargs],
) -> Iterator[dict[str, Any] | Any]

```

Stream graph steps for a single input.

PARAMETER	DESCRIPTION
<code>input</code>	The input to the graph. TYPE: <code>InputT</code> <code>Command</code> <code>None</code>
<code>config</code>	The configuration to use for the run. TYPE: <code>RunnableConfig</code> <code>None</code> DEFAULT: <code>None</code>
<code>context</code>	The static context to use for the run.  Added in version 0.6.0 TYPE: <code>ContextT</code> <code>None</code> DEFAULT: <code>None</code>
<code>stream_mode</code>	The mode to stream output, defaults to <code>self.stream_mode</code>. Options are: <code>"values"</code> : Emit all values in the state after each step, including interrupts. When used with functional API, values are emitted once at the end of the workflow. <code>"updates"</code> : Emit only the node or task names and updates returned by the nodes or tasks after each step. If multiple updates are made in the same step (e.g. multiple nodes are run) then those updates are emitted separately. <code>"custom"</code> : Emit custom data from inside nodes or tasks using <code>StreamWriter</code>. <code>"messages"</code> : Emit LLM messages token-by-token together with metadata for any LLM invocations inside nodes or tasks. - Will be emitted as 2-tuples <code>(LLM token, metadata)</code>.

PARAMETER	DESCRIPTION
	"checkpoints" : Emit an event when a checkpoint is created, in the same format as returned by <code>get_state()</code> . "tasks" : Emit events when tasks start and finish, including their results and errors. "debug" : Emit debug events with as much information as possible for each step. You can pass a list as the <code>stream_mode</code> parameter to stream multiple modes at once. The streamed outputs will be tuples of <code>(mode, data)</code> . See LangGraph streaming guide for more details. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> <code>None</code> DEFAULT: <code>None</code>
<code>print_mode</code>	Accepts the same values as <code>stream_mode</code> , but only prints the output to the console, for debugging purposes. Does not affect the output of the graph in any way. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> DEFAULT: <code>()</code>
<code>output_keys</code>	The keys to stream, defaults to all non-context channels. TYPE: <code>str</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_before</code>	Nodes to interrupt before, defaults to all nodes in the graph. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_after</code>	Nodes to interrupt after, defaults to all nodes in the graph. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>durability</code>	The durability mode for the graph execution, defaults to <code>"async"</code> . Options are: "sync" : Changes are persisted synchronously before the next step starts.

PARAMETER	DESCRIPTION
	"async": Changes are persisted asynchronously while the next step executes. "exit": Changes are persisted only when the graph exits. TYPE: <code>Durability None</code> DEFAULT: <code>None</code>
<code>subgraphs</code>	Whether to stream events from inside subgraphs, defaults to <code>False</code>. If <code>True</code>, the events will be emitted as tuples <code>(namespace, data)</code>, or <code>(namespace, mode, data)</code> if <code>stream_mode</code> is a list, where <code>namespace</code> is a tuple with the path to the node where a subgraph is invoked, e.g. <code>("parent_node:<task_id>", "child_node:<task_id>")</code>. See LangGraph streaming guide for more details. TYPE: <code>bool</code> DEFAULT: <code>False</code>

YIELDS	DESCRIPTION
<code>dict[str, Any] Any</code>	The output of each step in the graph. The output shape depends on the <code>stream_mode</code> .

meth `astream` `async`

```

astream(
    input: InputT | Command | None,
    config: RunnableConfig | None = None,
    *,
    context: ContextT | None = None,
    stream_mode: StreamMode | Sequence[StreamMode] | None = None,
    print_mode: StreamMode | Sequence[StreamMode] = (),
    output_keys: str | Sequence[str] | None = None,
    interrupt_before: All | Sequence[str] | None = None,
    interrupt_after: All | Sequence[str] | None = None,
    durability: Durability | None = None,
    subgraphs: bool = False,
    debug: bool | None = None,
    **kwargs: Unpack[DeprecatedKwargs],
) -> AsyncIterator[dict[str, Any] | Any]

```

Asynchronously stream graph steps for a single input.

PARAMETER	DESCRIPTION
<code>input</code>	The input to the graph. TYPE: <code>InputT</code> <code>Command</code> <code>None</code>
<code>config</code>	The configuration to use for the run. TYPE: <code>RunnableConfig</code> <code>None</code> DEFAULT: <code>None</code>
<code>context</code>	The static context to use for the run.  Added in version 0.6.0 TYPE: <code>ContextT</code> <code>None</code> DEFAULT: <code>None</code>
<code>stream_mode</code>	The mode to stream output, defaults to <code>self.stream_mode</code>. Options are: <code>"values"</code> : Emit all values in the state after each step, including interrupts. When used with functional API, values are emitted once at the end of the workflow. <code>"updates"</code> : Emit only the node or task names and updates returned by the nodes or tasks after each step. If multiple updates are made in the same step (e.g. multiple nodes are run) then those updates are emitted separately. <code>"custom"</code> : Emit custom data from inside nodes or tasks using <code>StreamWriter</code>. <code>"messages"</code> : Emit LLM messages token-by-token together with metadata for any LLM invocations inside nodes or tasks. - Will be emitted as 2-tuples <code>(LLM token, metadata)</code>. <code>"checkpoints"</code> : Emit an event when a checkpoint is created, in the same format as returned by <code>get_state()</code>. <code>"tasks"</code> : Emit events when tasks start and finish, including their results and errors. <code>"debug"</code> : Emit debug events with as much information as possible for each step.

PARAMETER	DESCRIPTION
	You can pass a list as the <code>stream_mode</code> parameter to stream multiple modes at once. The streamed outputs will be tuples of <code>(mode, data)</code>. See LangGraph streaming guide for more details. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> <code>None</code> DEFAULT: <code>None</code>
<code>print_mode</code>	Accepts the same values as <code>stream_mode</code>, but only prints the output to the console, for debugging purposes. Does not affect the output of the graph in any way. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> DEFAULT: <code>()</code>
<code>output_keys</code>	The keys to stream, defaults to all non-context channels. TYPE: <code>str</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_before</code>	Nodes to interrupt before, defaults to all nodes in the graph. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_after</code>	Nodes to interrupt after, defaults to all nodes in the graph. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>durability</code>	The durability mode for the graph execution, defaults to <code>"async"</code>. Options are: <code>"sync"</code>: Changes are persisted synchronously before the next step starts. <code>"async"</code>: Changes are persisted asynchronously while the next step executes. <code>"exit"</code>: Changes are persisted only when the graph exits. TYPE: <code>Durability</code> <code>None</code> DEFAULT: <code>None</code>

PARAMETER	DESCRIPTION
<code>subgraphs</code>	Whether to stream events from inside subgraphs, defaults to <code>False</code>. If <code>True</code>, the events will be emitted as tuples <code>(namespace, data)</code>, or <code>(namespace, mode, data)</code> if <code>stream_mode</code> is a list, where <code>namespace</code> is a tuple with the path to the node where a subgraph is invoked, e.g. <code>("parent_node:<task_id>", "child_node:<task_id>")</code>. See LangGraph streaming guide for more details. TYPE: <code>bool</code> DEFAULT: <code>False</code>

YIELDS	DESCRIPTION
<code>AsyncIterator[dict[str, Any] Any]</code>	The output of each step in the graph. The output shape depends on the <code>stream_mode</code> .

meth `invoke`

```
invoke(  
    input: InputT | Command | None,  
    config: RunnableConfig | None = None,  
    *,  
    context: ContextT | None = None,  
    stream_mode: StreamMode = "values",  
    print_mode: StreamMode | Sequence[StreamMode] = (),  
    output_keys: str | Sequence[str] | None = None,  
    interrupt_before: All | Sequence[str] | None = None,  
    interrupt_after: All | Sequence[str] | None = None,  
    durability: Durability | None = None,  
    **kwargs: Any,  
) -> dict[str, Any] | Any
```

Run the graph with a single input and config.

PARAMETER	DESCRIPTION
<code>input</code>	The input data for the graph. It can be a dictionary or any other type. TYPE: <code>InputT</code> <code>Command</code> <code>None</code>

PARAMETER	DESCRIPTION
<code>config</code>	The configuration for the graph run. TYPE: <code>RunnableConfig</code> <code>None</code> DEFAULT: <code>None</code>
<code>context</code>	The static context to use for the run.  Added in version 0.6.0 TYPE: <code>ContextT</code> <code>None</code> DEFAULT: <code>None</code>
<code>stream_mode</code>	The stream mode for the graph run. TYPE: <code>StreamMode</code> DEFAULT: <code>'values'</code>
<code>print_mode</code>	Accepts the same values as <code>stream_mode</code>, but only prints the output to the console, for debugging purposes. Does not affect the output of the graph in any way. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> DEFAULT: <code>()</code>
<code>output_keys</code>	The output keys to retrieve from the graph run. TYPE: <code>str</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_before</code>	The nodes to interrupt the graph run before. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_after</code>	The nodes to interrupt the graph run after. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>durability</code>	The durability mode for the graph execution, defaults to <code>"async"</code>. Options are: <code>"sync"</code>: Changes are persisted synchronously before the next step starts.

PARAMETER	DESCRIPTION
	"async": Changes are persisted asynchronously while the next step executes. "exit": Changes are persisted only when the graph exits.
	TYPE: <code>Durability None</code> DEFAULT: <code>None</code>
<code>**kwargs</code>	Additional keyword arguments to pass to the graph run.
	TYPE: <code>Any</code> DEFAULT: <code>{}</code>

RETURNS	DESCRIPTION
<code>dict[str, Any] Any</code>	The output of the graph run. If <code>stream_mode</code> is "values", it returns the latest output.
<code>dict[str, Any] Any</code>	If <code>stream_mode</code> is not "values", it returns a list of output chunks.

meth `ainvoke` `async`

```
ainvoke(
    input: InputT | Command | None,
    config: RunnableConfig | None = None,
    *,
    context: ContextT | None = None,
    stream_mode: StreamMode = "values",
    print_mode: StreamMode | Sequence[StreamMode] = (),
    output_keys: str | Sequence[str] | None = None,
    interrupt_before: All | Sequence[str] | None = None,
    interrupt_after: All | Sequence[str] | None = None,
    durability: Durability | None = None,
    **kwargs: Any,
) -> dict[str, Any] | Any
```

Asynchronously run the graph with a single input and config.

PARAMETER	DESCRIPTION
<code>input</code>	The input data for the graph. It can be a dictionary or any other type. TYPE: <code>InputT</code> <code>Command</code> <code>None</code>
<code>config</code>	The configuration for the graph run. TYPE: <code>RunnableConfig</code> <code>None</code> DEFAULT: <code>None</code>
<code>context</code>	The static context to use for the run.  Added in version 0.6.0 TYPE: <code>ContextT</code> <code>None</code> DEFAULT: <code>None</code>
<code>stream_mode</code>	The stream mode for the graph run. TYPE: <code>StreamMode</code> DEFAULT: <code>'values'</code>
<code>print_mode</code>	Accepts the same values as <code>stream_mode</code>, but only prints the output to the console, for debugging purposes. Does not affect the output of the graph in any way. TYPE: <code>StreamMode</code> <code>Sequence[StreamMode]</code> DEFAULT: <code>()</code>
<code>output_keys</code>	The output keys to retrieve from the graph run. TYPE: <code>str</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_before</code>	The nodes to interrupt the graph run before. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>interrupt_after</code>	The nodes to interrupt the graph run after. TYPE: <code>All</code> <code>Sequence[str]</code> <code>None</code> DEFAULT: <code>None</code>
<code>durability</code>	The durability mode for the graph execution, defaults to <code>"async"</code>. Options are:

PARAMETER	DESCRIPTION
	"sync": Changes are persisted synchronously before the next step starts. "async": Changes are persisted asynchronously while the next step executes. "exit": Changes are persisted only when the graph exits. TYPE: <code>Durability None</code> DEFAULT: <code>None</code>
<code>**kwargs</code>	Additional keyword arguments to pass to the graph run. TYPE: <code>Any</code> DEFAULT: <code>{}</code>

RETURNS	DESCRIPTION
<code>dict[str, Any] Any</code>	The output of the graph run. If <code>stream_mode</code> is "values", it returns the latest output.
<code>dict[str, Any] Any</code>	If <code>stream_mode</code> is not "values", it returns a list of output chunks.

meth `get_state`

```
get_state(config: RunnableConfig, *, subgraphs: bool = False) -> StateSnapshot
```

Get the current state of the graph.

meth `aget_state` `async`

```
aget_state(config: RunnableConfig, *, subgraphs: bool = False) -> StateSnapshot
```

Get the current state of the graph.

meth `get_state_history`

```
get_state_history(
```

```

    config: RunnableConfig,
    *,
    filter: dict[str, Any] | None = None,
    before: RunnableConfig | None = None,
    limit: int | None = None,
) -> Iterator[StateSnapshot]

```

Get the history of the state of the graph.

meth `aget_state_history` `async`

```

aget_state_history(
    config: RunnableConfig,
    *,
    filter: dict[str, Any] | None = None,
    before: RunnableConfig | None = None,
    limit: int | None = None,
) -> AsyncIterator[StateSnapshot]

```

Asynchronously get the history of the state of the graph.

meth `update_state`

```

update_state(
    config: RunnableConfig,
    values: dict[str, Any] | Any | None,
    as_node: str | None = None,
    task_id: str | None = None,
) -> RunnableConfig

```

Update the state of the graph with the given values, as if they came from node `as_node`. If `as_node` is not provided, it will be set to the last node that updated the state, if not ambiguous.

meth `aupdate_state` `async`

```

aupdate_state(
    config: RunnableConfig,
    values: dict[str, Any] | Any,
    as_node: str | None = None,
    task_id: str | None = None,
) -> RunnableConfig

```

Asynchronously update the state of the graph with the given values, as if they came from node `as_node`. If `as_node` is not provided, it will be set to the last node that updated the state, if not ambiguous.

meth bulk_update_state

```
bulk_update_state(  
    config: RunnableConfig, supersteps: Sequence[Sequence[StateUpdate]]  
) -> RunnableConfig
```

Apply updates to the graph state in bulk. Requires a checkpointer to be set.

PARAMETER	DESCRIPTION
<code>config</code>	The config to apply the updates to. TYPE: <code>RunnableConfig</code>
<code>supersteps</code>	A list of supersteps, each including a list of updates to apply sequentially to a graph state. Each update is a tuple of the form <code>(values, as_node, task_id)</code> where <code>task_id</code> is optional. TYPE: <code>Sequence[Sequence[StateUpdate]]</code>

RAISES	DESCRIPTION
<code>ValueError</code>	If no checkpointer is set or no updates are provided.
<code>InvalidUpdateError</code>	If an invalid update is provided.

RETURNS	DESCRIPTION
<code>RunnableConfig</code>	The updated config. TYPE: <code>RunnableConfig</code>

meth `abulk_update_state` `async`

```
abulk_update_state(  
    config: RunnableConfig, supersteps: Sequence[Sequence[StateUpdate]]  
) -> RunnableConfig
```

Asynchronously apply updates to the graph state in bulk. Requires a checkpointer to be set.

PARAMETER	DESCRIPTION
<code>config</code>	The config to apply the updates to. TYPE: <code>RunnableConfig</code>
<code>supersteps</code>	A list of supersteps, each including a list of updates to apply sequentially to a graph state. Each update is a tuple of the form <code>(values, as_node, task_id)</code> where <code>task_id</code> is optional. TYPE: <code>Sequence[Sequence[StateUpdate]]</code>

RAISES	DESCRIPTION
<code>ValueError</code>	If no checkpointer is set or no updates are provided.
<code>InvalidUpdateError</code>	If an invalid update is provided.

RETURNS	DESCRIPTION
<code>RunnableConfig</code>	The updated config. TYPE: <code>RunnableConfig</code>

meth `get_graph`

```
get_graph(config: RunnableConfig | None = None, *, xray: int | bool = False) -> Graph
```

Return a drawable representation of the computation graph.

meth `aget_graph` async

```
aget_graph(config: RunnableConfig | None = None, *, xray: int | bool = False)
Graph
```

Return a drawable representation of the computation graph.

meth `get_subgraphs`

```
get_subgraphs(
    *, namespace: str | None = None, recurse: bool = False
) -> Iterator[tuple[str, PregelProtocol]]
```

Get the subgraphs of the graph.

PARAMETER	DESCRIPTION
<code>namespace</code>	The namespace to filter the subgraphs by. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>recurse</code>	Whether to recurse into the subgraphs. If <code>False</code> , only the immediate subgraphs will be returned. TYPE: <code>bool</code> DEFAULT: <code>False</code>

RETURNS	DESCRIPTION
<code>Iterator[tuple[str, PregelProtocol]]</code>	An iterator of the <code>(namespace, subgraph)</code> pairs.

meth `aget_subgraphs` async

```
aget_subgraphs(
    *, namespace: str | None = None, recurse: bool = False
) -> AsyncIterator[tuple[str, PregelProtocol]]
```

Get the subgraphs of the graph.

PARAMETER	DESCRIPTION
<code>namespace</code>	The namespace to filter the subgraphs by. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>recurse</code>	Whether to recurse into the subgraphs. If <code>False</code> , only the immediate subgraphs will be returned. TYPE: <code>bool</code> DEFAULT: <code>False</code>

RETURNS	DESCRIPTION
<code>AsyncIterator[tuple[str, PregelProtocol]]</code>	An iterator of the <code>(namespace, subgraph)</code> pairs.

meth `with_config`

```
with_config(config: RunnableConfig | None = None, **kwargs: Any) -> Self
```



Create a copy of the Pregel object with an updated config.